The most usual way of setting (and passing) the locale would be to include it in URL params, as we did in the `I18n.locale = params[:locale]` *before_action* in the first example. We would like to have URLs like `www.example.com/books?locale=ja` or `www.example.com/ja/books` in this case.
This approach has almost the same set of advantages as setting the locale from the domain name: namely that it's RESTful and in accord with the rest of the World Wide Web. It does require a little bit more work to implement, though.

Getting the locale from `params` and setting it accordingly is not hard; including it in every URL and thus **passing it through the requests** is. To include an explicit option in every URL, e.g. `link_to(books_url(locale: I18n.locale))`, would be tedious and probably impossible, of course.
Rails contains infrastructure for "centralizing dynamic decisions about the URLs" in its `ApplicationController#default_url_options`, which is useful precisely in this scenario: it enables us to set "defaults" for `url_for` and helper methods dependent on it (by implementing/overriding this method).
We can include something like this in our `ApplicationController` then:

```
# app/controllers/application_controller.rb
def default_url_options(options = {})
  { locale: I18n.locale }.merge options
end
```

Every helper method dependent on `url_for` (e.g. helpers for named routes like `root_path` or `root_url`, resource routes like `books_path` or `books_url`, etc.) will now **automatically include the locale in the query string**, like this: `http://localhost:3001/?locale=ja`.
You may be satisfied with this. It does impact the readability of URLs, though, when the locale "hangs" at the end of every URL in your application. Moreover, from the architectural standpoint, locale is usually hierarchically above the other parts of the application domain: and URLs should reflect this.

You probably want URLs to look like this: `www.example.com/en/books` (which loads the English locale) and `www.example.com/nl/books` (which loads the Dutch locale). This is achievable with the "overriding `default_url_options`" strategy from above: you just have to set up your routes with `scoping` option in this way:

```
# config/routes.rb
scope "/:locale" do
  resources :books
end
```

Now, when you call the `books_path` method you should get `"/en/books"` (for the default locale). An URL like `http://localhost:3001/nl/books` should load the Dutch locale, then, and following calls to `books_path` should return `"/nl/books"` (because the locale changed).
If you don't want to force the use of a locale in your routes you can use an optional path scope (denoted by the parentheses) like so:

```
# config/routes.rb
scope "(:locale)", locale: /en|nl/ do
  resources :books
end
```

With this approach you will not get a `Routing Error` when accessing your resources such as `http://localhost:3001/books` without a locale. This is useful for when you want to use the default locale when one is not specified.
Of course, you need to take special care of the root URL (usually "homepage" or "dashboard") of your application. An URL like `http://localhost:3001/nl` will not work automatically, because the `root to:`